

fwforge

Palo Alto → FortiGate Conversion

Capability overview and comparison with FortiConverter

An open, clean-room PAN-OS → FortiOS converter. Prepared for Fortinet engineering review · June 2026 · fwforge v0.49.0

Contents

1. Executive summary

fwforge is a transparent, fully local tool that converts Palo Alto PAN-OS configurations — firewall XML, set/display-set format, and Panorama exports — into clean, restorable FortiOS configuration. It exists to make migrating off Palo Alto and onto FortiGate fast, auditable, and trustworthy.

- **Clean-room.** It contains no Fortinet or FortiConverter code, and no licensed FortiGuard data files. It targets the public FortiOS CLI only.
- **An on-ramp to FortiGate.** Every conversion is a Palo Alto estate moving onto FortiOS. The tool's purpose is to remove migration friction from that path.
- **Parity on the core, ahead on assurance.** It matches FortiConverter on the object / policy / NAT / routing / VPN conversion, and goes beyond it on output quality: every emitted config is schema-certified against the exact target firmware build, is deterministic and git-diffable, and carries per-line provenance back to the source line.

The conversions and category data in this document were **verified live against a FortiGate-601F running FortiOS 8.0.0 build0167** — FortiGuard web categories and file types were read from the device, and the emitted output was schema-certified clean (0 unknown tables, 0 unknown attributes).

2. What makes it different

Six properties distinguish fwforge from a typical converter. They are the reasons the output can be trusted on a production cutover.

- **Schema-certified output.** Every section and attribute is validated against the exact target firmware's CLI schema, fetched live from the device (one read-only API call) or from cache. Unknown table = error; unknown attribute = warning. Nothing reaches a device unverified.
- **Deterministic and diffable.** No timestamps, no random ordering. Re-running the same input yields byte-identical output, so a reviewer can diff two conversions in git and see exactly what changed.
- **Per-line provenance and a real report.** Every converted object records its source file and line. Findings are a structured report (Markdown + JSON + HTML) with severity and location — not warnings buried as comments in the config. A non-zero exit code is returned on errors.

- **Version-delta awareness.** fwforge compares source and target FortiOS versions and reports removed or introduced features, auto-fixed attribute/section renames, and silent default-flips — in both upgrade and downgrade directions.
- **Never silently broadens or drops.** Anything it cannot faithfully convert is flagged, never approximated into something more permissive. For example, a custom udp/53 service is not collapsed onto the FortiOS built-in DNS service, which is tcp+udp/53.
- **Local and target-model aware.** It runs as a single local package with no cloud dependency, and maps interfaces against the destination model's real port inventory (read from a backup or model table) using a FortiOS-style faceplate.

3. PAN-OS → FortiGate capability matrix

Everything fwforge converts from a Palo Alto source. Anything outside this table is reported as a finding, not dropped silently.

Domain	What fwforge converts	Notes
Input formats	PAN-OS firewall XML and set / display-set format; Panorama exports	One unified parser, auto-detected. Panorama template-merged running-configs handled.
Panorama & multi-vsyz	Device-group selection; shared + pre/post rulebases merged in PAN evaluation order; optional template for network config; each vsyz → its own FortiOS VDOM	Cross-vsyz references flagged.
Interfaces	Physical, aggregate (LAG), Layer-3 subinterfaces (VLAN), loopback, tunnel	Mapped to the destination model's real ports via a faceplate UI with positional auto-guess; promote-a-port-to-LAG and “do not map” supported.
Aggregates / LAG	PAN aggregate-ethernet → FortiOS 802.3ad aggregate (type aggregate + member ports + lacp-mode)	LACP mode read from source; emitted before the VLANs that ride it so the script loads in order.
Zones	PAN security zones → FortiOS zones	Emitted with intrazone allow to preserve PAN's default same-zone behavior; flagged for tightening.
Addresses	host / subnet / range / FQDN; IPv4 and IPv6 (address6)	Names sanitized to FortiOS limits with every reference remapped.
Address groups	Nested groups with single-family enforcement	Mixed-family members handled per FortiOS rules.
Services	Custom TCP/UDP (port + source-port ranges), service groups, predefined service-http/https	Mapped to FortiOS built-ins only on exact semantic match — never broadened.
App-ID	App-IDs → application-control category profiles; App-IDs → policy service (standard ports); application-groups flattened to leaves	Category-level (see §6). App-IDs that match a FortiOS built-in service are emitted as that named service.
URL filtering	PAN url-filtering profiles → FortiOS webfilter (FortiGuard category) profiles	Category IDs verified live. PAN action → ftgd-wf action (block; alert→monitor; continue→warning; override→authenticate).
File blocking	PAN file-blocking profiles → FortiOS file-filter profiles	File types verified live. PAN action → block / log-only / warning.

SSL inspection	webfilter / file-filter policies attached with the built-in certificate-inspection profile	SNI-based; no CA rollout. Switch to deep-inspection for full HTTPS content control.
Security policies	PAN rules → FortiOS firewall policy: zones, addresses, services, action, logging, disable state, source/destination negation	Per-rule selection (exclude rules); schedules flagged; application-default handled.
NAT	Source NAT (interface PAT) → policy NAT or central-snat-map; destination/static NAT → VIP (+ port-forward); subnet 1:1 DNAT → range VIP	NAT mode selectable: policy or central.
Routing	Static routes (egress inferred when omitted); BGP (AS, router-id, neighbors, networks, redistribute); OSPF (areas, networks, passive); IPv6 routes	The route table also drives policy dstintf inference (longest-prefix-match).
IPsec VPN	Site-to-site → route-based phase1/phase2-interface + tunnel routes + bidirectional policies; IKE crypto, proposals, PFS, IKEv2	Encrypted / exported PSKs detected → placeholder + error (never a silently broken secret).
Output hygiene	Prune unreferenced objects (iterative); merge duplicate objects; split interface pairs; include/exclude rules	Acts on the output, not just reports.
Output packaging	config-all.txt + per-branch script files; findings embedded as # comments	Deterministic ordering; restore-safe.
FortiManager	Optional JSON-RPC import bundle: object creates + a policy package for an ADOM	
Assurance	Schema certification; version-delta scan; Markdown / JSON / HTML report; XML coverage map (% of source read)	See §2 and §5.

4. Security-profile conversion

PAN security profiles split into two groups: those that are category- or type-level (faithfully convertible in a clean-room tool) and those that are signature/engine-level (which are flagged, not guessed).

Converted

- **App-ID → Application Control.** A rule's App-IDs become a FortiOS application-control profile (FortiGuard categories) and, in parallel, a port-based service from the apps' standard ports — so the policy is both port-based and app-aware. Application-groups are expanded to their leaf apps.
- **URL filtering → Web Filter.** Each PAN url-filtering profile becomes a FortiOS FortiGuard-category webfilter profile. PAN per-category actions map to ftgd-wf actions; one PAN category can expand to several FortiGuard categories (e.g. alcohol-and-tobacco → Alcohol + Tobacco). Profiles are deduplicated and attached to every policy that referenced them.
- **File blocking → File Filter.** Each PAN file-blocking profile becomes a FortiOS file-filter profile; PAN file types map to FortiOS file types over HTTP/FTP/SMTP/IMAP/POP3/MAPI/CIFS/SSH.
- **SSL inspection.** Policies carrying a web or file profile are attached to the built-in certificate-inspection profile (SNI-based, no certificate rollout). Deep-inspection is recommended in the report for full HTTPS content control.

Flagged, not converted (by design)

Antivirus, Anti-Spyware, Vulnerability Protection, WildFire, and Data Filtering are signature/engine-level and are reported per rule for manual attachment rather than approximated. PAN risk-level URL buckets (high/medium/low-risk) and any unmapped category or file type are likewise flagged — never silently dropped.

5. Output quality and assurance

- **Schema certification.** The emitted CLI is parsed and checked table-by-table, attribute-by-attribute, against the target build's schema. In this review a full PAN→FortiGate conversion certified clean against a live 601F (FortiOS 8.0.0 build0167): 0 unknown tables, 0 unknown attributes.
- **Deterministic output.** No timestamps or nondeterministic ordering, so successive runs diff cleanly.
- **Structured findings.** Markdown, JSON, and HTML reports, each finding tagged with severity, area, and source file:line. The HTML report is print-to-PDF friendly for hand-off.
- **Coverage map.** The parser reports the percentage of source values it read and flags any unread subtree, so reviewers can see what was and was not considered.

6. Scope and limitations (stated plainly)

Engineers should know exactly where the tool stops. Each of these is surfaced in the report; nothing fails silently.

- **Category-level App-ID and URL mapping.** fwforce maps to FortiOS application-control and FortiGuard web categories, not per-application or per-URL signatures. The PAN-app → FortiGuard-signature-ID table is licensed and cannot be used in a clean-room tool, so this is coarser than FortiConverter's licensed mapping. With a Fortinet-sanctioned mapping, fwforce could emit signature-level profiles — a natural collaboration point.

- **Engine-level profiles.** Antivirus, IPS (Anti-Spyware / Vulnerability), WildFire, and Data Filtering are not converted (see §4).
- **Complex NAT.** Common source and destination NAT convert; exotic twice-NAT idioms are flagged for manual handling.
- **Remote-access VPN.** Site-to-site IPsec converts; GlobalProtect / remote-access is not converted.

7. fwforge vs FortiConverter

A feature-and-quality comparison for the Palo Alto → FortiGate path. Both tools convert the core object/policy/NAT/routing model; the differences are in coverage, granularity, and output assurance.

Dimension	fwforge	FortiConverter
Source coverage	Palo Alto (XML, set, Panorama, multi-vsyz)	Palo Alto plus 15+ other vendors — broader
App-ID / URL granularity	Category-level (public IDs, clean-room)	Signature-level (licensed FortiGuard table) — finer
Output validation	Schema-certified against the exact target firmware build	Not certified against the live target schema
Determinism	Byte-deterministic, git-diffable	Not deterministic
Provenance / findings	Per-line source provenance; md/JSON/HTML report; non-zero exit on error	Warnings as comments in config-all.txt
Version-delta scan	Yes — bidirectional feature / rename / default-flip	No
Object hygiene	Prunes, merges, dedups (acts on output)	Reports; discard is opt-in
Routing-aware dstintf	Longest-prefix-match from source routes	Falls back to any when missing (per docs)
Deployment	Single local package, fully offline	Heavier install; FGT→FGT path runs in cloud
FortiManager output	JSON-RPC import bundle (objects + package)	Yes — mature
Support model	Self-hosted / community	Official Fortinet support + service SLA

Where FortiConverter leads: vendor breadth, signature-level App-ID/URL mapping, official support, and product maturity. **Where fwforge leads:** output assurance (schema certification), transparency, determinism, provenance, version-delta awareness, active hygiene, and fully-local operation.

8. Example output

A PAN physical port carrying VLANs, promoted to a LAG, with a url-filtering profile — as emitted (and schema-certified clean) by fwforge:

```
config system interface
```

```

edit "lag-uplink"
set type aggregate
set member "port5" "port6"
set lacp-mode active
next
edit "vlan100"
set type vlan
set interface "lag-uplink" # rides the LAG; emitted after it
set vlanid 100
next
end
config webfilter profile
edit "wf-url-strict"
config ftgd-wf
config filters
edit 1
set category 26 # Malicious Websites
set action block
next
end
end
next
end
config firewall policy
edit 1
set name "trust-to-untrust"
set utm-status enable
set ssl-ssh-profile "certificate-inspection"
set webfilter-profile "wf-url-strict"
next
end

```

Appendix: verification methodology

To keep category and type mappings honest, fwforge's PAN-OS conversion data was verified against a live FortiGate rather than from documentation alone:

- **FortiGuard web categories** (the ~93 IDs used by URL-filtering) were read from the device at `/api/v2/monitor/webfilter/fortiguard-categories`.
- **File-filter file types** were read from the device's antivirus.filetype data source.
- **The emitted configuration** was run through fwforge's own schema certifier against the device's CLI schema and returned 0 unknown tables and 0 unknown attributes.

Reference device: FortiGate-601F, FortiOS 8.0.0 build0167.